



Université de
Technologie de
Compiègne

LO17/AI31 : Indexation et Recherche d'Information

Rapport de Projet RAG – Cinéma Français

Membre du groupe (+pourcentages participation) :

Bouaziz Selima

Lozac'H Mael

Marko-Lafon Clothilde

Peter-Hemon Edeline

Groupe :

D1-G4



Sommaire



Partie 1

Introduction

Ce projet consiste à implémenter un RAG sur le sujet de notre choix. Nous avons donc décidé de partir sur le cinéma français. Pour cela, nous avons choisi de nous appuyer sur la liste Wikipedia des plus gros succès français au box office mondial (voir annexe), ainsi que sur les données des pages Wikipedia dédiées à chacun de ces films.

Nous avons utilisé le modèle Gemini de Google pour LLM, et suivi le notebook officiel de Google Gemini pour coder notre RAG.

1.1 Architecture générale

Nous avons divisé le code en plusieurs fichiers python afin de faciliter la compréhension, la maintenance ainsi que les tests. Le système suit l'arborescence suivante :

[main.py]

- [crawler.py] → Récupération des URLs des films
- [retriever.py] → Construction de la base vectorielle
 - [film_parser.py] → Extraction du contenu des pages Wikipédia
 - [database.py] → Stockage SQL des données structurées
- [generator.py] → Génération des réponses avec Gemini

Les clés API Google Gemini sont elles stockées dans un fichier dédié secrets.py pour ne pas les exposer dans le code principal.



Partie 2

Description des modules

2.1 Module principal (main.py) :

Ce fichier est le point d'entrée du programme, c'est lui qui gère l'exécution des 4 autres modules dans l'ordre, correspondants aux étapes suivantes :

- Configuration : On initialise la clé API Google Gemini et l'agent utilisateur.
- Crawling : On récupère les URLs des pages Wikipedia de chacun des films via la page initiale qui les liste.
- Indexation : On construit la base de vectorielle à partir de ces pages
- Interaction : Correspond à la boucle de questions/réponses entre le système et l'utilisateur. L'utilisateur pose une question, et le système affiche la réponse générée par Gemini. Cette boucle continue jusqu'à ce que l'utilisateur l'interrompe (Contrôle C).

2.2 Module de crawler (crawler.py) :

Ce fichier comporte deux fonctions :

- `get_movie_titles()` : cette première fonction extrait le titre des films depuis la page wikipedia initiale. Pour cela elle utilise la librairie BeautifulSoup pour parser le contenu HTML, puis cible les balises `<table class= 'wikitable'>` et en récupère la deuxième colonne de chaque ligne, soit le titre d'un film. Elle nettoie ensuite les références de type [1] inutiles avec un regex `(r'\[.*?\]')`.
- `load_movie_urls()` : cette seconde fonction convertit chaque titre en URL Wikipedia en utilisant l'API Wikipedia que nous avons récupéré. Elle permet de gérer plusieurs cas spécifiques afin d'éviter les bugs ou erreurs : elle respecte les limites de l'API en faisant une pause d'une seconde toutes les 10 requêtes, filtre les pages inexistantes et gère les cas d'homonymie (par exemple le film 'Lucy' et le personnage historique du même nom). Le système essaie d'abord le titre exacte, puis cherche 'titre (film)' si ça échoue.

Nous avons faits plusieurs choix d'implémentations qui permettent le bon fonctionnement de ce module. Par exemple :

- L'utilisation d'un set pour les titres permet d'éviter les doublons entre les deux tableaux de films.
- Le `auto-suggest = False` permet d'éviter les suggestions automatiques de Wikipedia qui pourraient mener à des pages non voulues.

2.3 Module d'extraction (film_parser.py)

Ce fichier est le cœur de l'extraction des données. C'est lui qui nous permet de rendre notre base de données suffisamment complète pour pouvoir répondre aux questions de l'utilisateur.

Contrairement au notebook qui automatise ce processus avec UnstructuredURLLoader, nous avons fait un parsing personnalisé au format des pages Wikipedia. Ici on analyse la page de chaque film récupérée avec le module précédent, et deux sorties différentes sont produites en fonction de deux types de données (textuelles et structurées). L'élément renvoyé suit donc un format `tuple[str, dict]` :

- Le `str` contient un texte continu qui sert à la recherche vectorielle, contenant l'introduction si elle existe, ainsi que le synopsis, l'accueil et l'analyse.



- Le dictionnaire contient des données structurées à filtrer dans un second temps avec du SQL, contenant la fiche technique, la distribution et les distinctions.

Nous avons choisi de garder ces éléments là afin d'avoir le plus d'informations utiles pour répondre aux questions, sans trop surcharger la base de données.

Ces éléments sont parcourus en utilisant de nouveau la librairie BeautifulSoup afin de récupérer dans les éléments HTML de la page les sections `<h2>` pour organiser le contenu, puis le texte des paragraphes `<p>` et des listes `<ul>` ou `<ol>`. Pour les sections SQL, on stocke temporairement les éléments bruts dans `sql_elems` pour un traitement séparé. Aussi, les références et espaces multiples sont nettoyés grâce à la fonction `clean_wiki_refs()`.

Les distinctions sont organisée par sous-catégories (par exemple César) sous un format avec crochets (`[Catégorie]`).

Ces fonctions suivent également l'évolution de Wikipedia, elles fonctionnent avec l'ancien comme avec le nouveau format (depuis 2023).

2.4 Module de base de données SQL (`database.py`) :

Ce fichier gère le stockage des données structurées renvoyées sous forme de dictionnaire dans le module précédent. On traite les données suivantes :

- films : le tableau maître avec les identifiants uniques et les URLs
- fiche_technique : on les stocke sous formes de paires clé/valeur (par exemple «Réalisateur» : «Duccio Tessari») de manière flexible, chaque film peut avoir des champs différents.
- distribution : la liste des acteurs et leurs rôles
- distinctions : la liste des récompenses, où le contexte est récupéré avec `[Catégorie]`.

On a deux fonctions dans ce module :

- La fonction `init_db()`, qui crée les tables si elles n'existent pas déjà, puis retourne une connexion SQLite. L'utilisation de SQLite plutôt par exemple de PostgreSQL est utile car elle ne nécessite aucun serveur.
- La fonction `save_film()` qui vérifie que l'URL n'existe pas déjà, puis insère le film et ses données associées, et utilise `executemany()` pour réaliser un bulk insert efficace (commande SQL servant à charger une grande quantité de données).

Les vérifications d'existence, comme le `UNIQUE` sur l'attribut `url` ou le `IF NOT EXISTS`, permettent de relancer l'indexation en évitant de recréer la base de données ou d'y insérer des doublons.

2.5 Module de retriever (`retriever.py`) :

Ce fichier construit la base vectorielle qui permettra la recherche sémantique. Il suit les étapes suivantes :

- D'abord une vérification de persistance. Si Chroma existe déjà, on le charge directement.
- Ensuite l'extraction. Pour chaque URL, on extrait le texte et les données avec `film_parser.py`.
- Le découpage : on divise le texte tous les 1000 caractères, avec un chevauchement de 200 (données venant du notebook).
- Les embeddings : On convertit ces parties de texte (appelées chunks) en vecteurs numériques en utilisant Gemini.
- Pour finir, le stockage. On sauvegarde les vecteurs et données dans Chroma.

Pour gérer les limites API, Gemini impose une limite de 100 requêtes par minute. Pour être sûr la respecter, le système traite les embeddings en lots de 90 et fait une pause d'une minute entre chacun de ces lots. Il évite comme ça les erreurs 429 (too many requests).



Un logging d'avancement est afficher toutes les 10 pages pour qu'on puisse vérifier l'avancement..

2.6 Module de génération (generator.py) :

Ce fichier est le plus important car il définit la chaîne RAG, avec pour composants :

- Retriever, qui recherche des cinq chunks les plus pertinents dans la base vectorielle
- format_docs, qui concatène les chunks trouvés en une seule chaîne.
- Prompt, qui intègre le contexte et la question.
- LLM, ici Gemini 2.0 Flash, qui génère la réponse.
- OutputParser, qui extrait le texte de la réponse.

Ce module suit entièrement le notebook, hormis l'adaptation du code pour le cinéma français.



Partie 3

Conclusion



Partie 4

Annexe

Le site wikipedia sur lequel ce projet se base : https://fr.wikipedia.org/wiki/Liste_des_plus_gros_succ%C3%A8s_fran%C3%A7ais_au_boxoffice_mondial